Utilisation de WebSockets

Plan

I. INTRODUCTION ET FONDAMENTAUX DE WEBSOCKET ---- 1

I.1. VUE D'ENSEMBLE DE LA COMMUNICATION CLIENT/SERVEUR ---- 1
I.2. LE HANDSHAKE WEBSOCKET ---- 1
I.3. BIBLIOTHEQUES JAVA (SERVEUR) ET ANDROID (CLIENT) ---- 2
I.4. NOTE IMPORTANTE POUR ANDROID ---- 2

II. APPLICATION CHAT ---- 3

II.1. SERVEUR CHAT ---- 3
II.2. CLIENT CHAT ---- 4

III. EXEMPLE D'ENVOI ET DE RECEPTION D'UN OBJET JSON ---- 5

I. Introduction et Fondamentaux de WebSocket

I.1. Vue d'ensemble de la Communication Client/Serveur

Pour qu'une application **Android (client)** communique en temps réel avec une application **serveur (Java)**, le protocole **WebSocket** est le choix idéal.

- **Communication HTTP Traditionnelle** : Repose sur le modèle **Requête-Réponse** (Half-Duplex). Le client envoie une requête, le serveur répond, puis la connexion est souvent fermée. Pour les mises à jour en temps réel, cela nécessite du **polling** (le client interroge le serveur périodiquement), ce qui est inefficace et gourmand en ressources.
- **Protocole WebSocket (WS)** : Fournit un canal de communication **bidirectionnel** (Full-Duplex) et **persistant** sur une seule connexion TCP. Après un processus initial de "poignée de main" (handshake) basé sur HTTP, la connexion reste ouverte, permettant au serveur d'envoyer des données au client de manière asynchrone (mode **Push**).

I.2. Le Handshake WebSocket

L'établissement de la connexion WebSocket est le seul moment où le protocole HTTP est utilisé.

1. **Requête du Client (HTTP Upgrade)** : Le client Android initialise une requête HTTP standard de type GET vers l'URL du serveur, mais inclut des en-têtes spécifiques pour demander la mise à niveau du protocole :
    - Upgrade: websocket : Indique l'intention de passer au protocole WebSocket.
    - Connection: Upgrade : Confirme l'intention de mise à niveau de la connexion.
    - Sec-WebSocket-Key : Une clé aléatoire utilisée par le serveur pour calculer un code de sécurité.
    - L'URL utilise un schéma spécifique : **ws://** pour une connexion non sécurisée ou **wss://** pour une connexion sécurisée (WebSocket Secure, via TLS/SSL).
2. **Réponse du Serveur (HTTP 101 Switching Protocols)** : Si le serveur supporte WebSocket, il répond avec :
    - Statut HTTP **101 Switching Protocols**.

- Les en-têtes Upgrade: websocket et Connection: Upgrade.
- Sec-WebSocket-Accept : La clé calculée par le serveur à partir de la clé du client, prouvant qu'il comprend le protocole.

3. **Connexion Établie** : Une fois la réponse 101 reçue, la connexion TCP sous-jacente est détournée de HTTP à WebSocket. Le canal est désormais **full-duplex** et les échanges se font via des **trames WebSocket**.

I.3. Bibliothèques Java (Serveur) et Android (Client)

Puisque nous utilisons Java des deux côtés, il existe plusieurs bibliothèques populaires :

Côté	Technologie/Bibliothèque	Remarques
Serveur Java	**JSR 356 (Jakarta EE)**	L'API standard pour Java EE (ex: Tomcat, WildFly).
	Spring Boot	Utilisation de **Spring WebSockets**, souvent avec STOMP pour un niveau d'abstraction supérieur.
	Java-WebSocket	Une bibliothèque légère, utile pour des serveurs stand-alone ou sur des cartes embarquées (si environnement Java SE).
Client Android	**OkHttp**	Bibliothèque très utilisée, inclut une implémentation WebSocket (OkHttpClient).
	Java-WebSocket	La même bibliothèque côté client Android.

Exporter vers Sheets

I.4. Note importante pour Android

- **L'accès réseau doit être effectué sur un thread séparé de l'interface utilisateur.**
- **L'autorisation d'accès à Internet est obligatoire dans le fichier AndroidManifest.xml :<uses-permission android**:name="android.permission.INTERNET" />
- **Configuration du Client Android (OkHttp) :** Nous utiliserons OkHttp comme exemple pour sa popularité et sa simplicité. Ajouter de la Dépendance (dans build.gradle au niveau de l'application) :

```
implementation 'com.squareup.okhttp3:okhttp:4.12.0' // Vérifiez la dernière version
```

II. Application chat

II.1. Serveur chat

```
//Fichier pom.xml (on utilise Maven)
    <dependencies>
        <dependency>
            <groupId>org.java-websocket</groupId>
            <artifactId>Java-WebSocket</artifactId>
            <version>1.6.0</version>
        </dependency>
    </dependencies>
// Fichier DesktopServer.kt
package org.example
import org.java_websocket.WebSocket
import org.java_websocket.handshake.ClientHandshake
import org.java_websocket.server.WebSocketServer
import java.net.InetSocketAddress
class DesktopServer(port: Int) : WebSocketServer(InetSocketAddress(port)) {

    override fun onOpen(conn: WebSocket, handshake: ClientHandshake) {
        println("Nouvelle connexion de: ${conn.remoteSocketAddress}")
        // Envoyer un message de bienvenue au client
        conn.send("Bienvenue! Vous êtes connecté au serveur Desktop.")
    }

    override fun onClose(conn: WebSocket, code: Int, reason: String,
remote: Boolean) {
        println("Déconnexion de: ${conn.remoteSocketAddress} Code: $code")
    }

    override fun onMessage(conn: WebSocket, message: String) {
        println("Message reçu de ${conn.remoteSocketAddress}: $message")
        // Renvoyer le message reçu à tous les clients
        broadcast(message)
    }

    override fun onError(conn: WebSocket?, ex: Exception?) {
        println("Erreur sur la connexion:
${conn?.remoteSocketAddress}:${ex?.message}")
    }

    override fun onStart() {
        println("Serveur WebSocket démarré sur le port $port")
        connectionLostTimeout = 100 // Optionnel: pour ajuster la gestion
des connexions perdues
    }

    companion object {
        @JvmStatic
        fun main(args: Array<String>) {
            val port = 8887
            val server = DesktopServer(port)
            server.start() // Démarrer le serveur
        }
    }
}
```

II.2. Client chat

```
//Fichier AndroidManifest.xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.INTERNET" />
    <application
        android:usesCleartextTraffic="true"
//Fichier build.gradle

dependencies {
      implementation("com.squareup.okhttp3:okhttp:4.12.0")
// Fichier MainActivity.kt
package org.example // Adaptez selon votre package réel

class MainActivity : AppCompatActivity() {
    private lateinit var btnConnecter: Button
    private lateinit var btnEnvoyer: Button
    private lateinit var btnDeconnecter: Button

    private val client = OkHttpClient()
    private var webSocket: WebSocket? = null

    // Remplacez par l'adresse IP de votre machine desktop et le port
    private val SERVER_URL = "ws://172.16.21.232:8887"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }
        initGraphique()
    }
    private fun initGraphique() {
        btnConnecter = findViewById(R.id.btnConnecter)
        btnEnvoyer = findViewById(R.id.btnEnvoyer)
        btnDeconnecter = findViewById(R.id.btnDeconnecter)
        ajouterEcouteurs()
    }
    private fun ajouterEcouteurs() {
        btnConnecter.setOnClickListener { connect() }
        btnEnvoyer.setOnClickListener { sendMessage("Ok http") }
        btnDeconnecter.setOnClickListener { disconnect() }
    }
    fun connect() {
        val request = Request.Builder().url(SERVER_URL).build()

        // Android exige que les opérations réseau soient sur un thread en arrière-plan.
        // OkHttp gère cela en interne pour la création du WebSocket.
        webSocket = client.newWebSocket(request, object : WebSocketListener() {
            override fun onOpen(webSocket: WebSocket, response: Response) {
                Log.i("Client", "Client connecté au serveur Desktop.")
                sendMessage("Bonjour du client Android!")
            }

            override fun onMessage(webSocket: WebSocket, text: String) {
                Log.i("Client", "Message reçu du serveur: $text")
              // Mettre à jour l'interface utilisateur (UI) sur le Main Thread si besoin :
                // runOnUiThread { /* ... */ }
            }
```

```
            override fun onClosing(webSocket: WebSocket, code: Int, reason: String) {
                webSocket.close(1000, null)
                Log.i("Client", "Connexion fermée par le serveur.")
            }

         override fun onFailure(webSocket: WebSocket, t: Throwable, response: Response?) {
                t.printStackTrace()
                System.err.println("Échec de la connexion: ${t.message}")
            }
        })
    }

    fun sendMessage(message: String) {
        webSocket?.send(message)
    }

    fun disconnect() {
        webSocket?.close(1000, "Fermeture normale")
    }
}
```

III. Exemple d'envoi et de réception d'un objet JSON

```
// Côté client (en Kotlin)
val jsonC = JSONObject().apply {
    put("bras", 1)
    put("valeur", 30)
}

val commande = jsonC.toString()
Log.i("Commande:", commande)
sendMessage(commande)
//Coté serveur
/*
Pom.xml
        <dependency>
            <groupId>org.json</groupId>
            <artifactId>json</artifactId>
            <version>20250517</version>
        </dependency>
module-info.java
        requires org.json;
 */
// Côté serveur ou récepteur (en Kotlin)
val jsonCR = JSONObject(commande)

val bras = jsonCR.getInt("bras")
val valeur = jsonCR.getInt("valeur")

println("Bras:$bras\nValeur:$val valeur")
```

Références :

[1] https://www.jmdoudoux.fr/java/dej/chap-net.htm

[2] Gemini